Seasonal Living Guide: Reproducible Regional Temperature Comparison with SQL Analytics and Monthly Reference Information

Team 05 · English paper draft · November 12, 2026 (Asia/Seoul)

Nunnalin Heimvichit (2371001), Jieun Hwang (2376329), Sumin Cho (2376274), Koo Daon (2476009), Jihee Choi (2276337)

The names above reproduce the supplied team roster. The confirmed assignment gives each member 20% technical, report, paper and presentation responsibility; actual personal review remains NOT_RUN. Implementation and draft preparation used one sequential Codex session; this draft does not claim that automatically generated commits establish personal manual authorship.

Abstract

Historical temperature data can support understandable comparisons when data grain, denominators and temporal boundaries remain explicit. We present Seasonal Living Guide, an Apache/PHP/MariaDB application comparing five requested locations in Korea over 2023–2025. The database contains 5,480 NASA POWER daily temperature records, 72 dated solar-term references and 12 monthly ingredient-guide items in six relational tables. Four separate interfaces implement complex grouping, hierarchical exploration with SQL roll-up, condition-based dense ranking and continuous 7/30-day moving averages. Users enter dates, locations and temperature limits through validated forms, while native prepared statements keep values separate from SQL structure. Saved plans use transactional parent/association writes and PHP Session application across pages. An independent Decimal oracle checks actual source data, while synthetic fixtures test ties, precision, incomplete windows and intermediate foreign-key failures. Fresh Apache/MariaDB and official Linux XAMPP installations passed initialization, reset, dump-only restoration and offline HTTP workflows. We report correctness and reproducibility evidence rather than scalability or recommendation-accuracy claims. Calendar and food information remain separate references and do not change temperature denominators.

Keywords: historical temperature; relational database; SQL window functions; dense ranking; transaction; reproducibility.

1. Introduction

A question such as “Which location most often met my temperature limits during the same historical month?” appears simple, yet an implementation can answer it incorrectly in several ways. Filtering away nonmatching days before calculating a percentage makes every retained group appear fully satisfactory. Ranking rounded display percentages can merge genuinely different rates, while adding a location identifier to a ranking window can destroy genuine ties. Selecting a displayed year before calculating a moving average loses necessary prior-year history. These errors can remain hidden behind a plausible interface.

Seasonal Living Guide addresses these issues through explicit daily records, reusable validity rules, database-side analytics and independent verification. It compares Seoul, Busan, Daegu, Gwangju and Jeju using identical requested dates and user-chosen limits. Separate menus expose four analysis families instead of disguising one table as several features. Users can also store common comparison conditions and later restore them to a PHP session.

The seasonal component provides contextual reference information, rather than a temperature-derived recommendation model. Solar terms are dated calendar occurrences; ingredient items are historical monthly editorial selections. Neither source establishes local harvest dates or weather-dependent availability. Keeping these sources separate avoids both analytical row multiplication and overstated interpretation.

The contribution of this implementation is a small, reproducible integration of relational modeling, dynamic SQL analytics, transactional web behavior and boundary-focused verification. It is not a distributed-processing system or a measured usability intervention. The intended educational setting values transparent query behavior, correct CRUD and repeatable installation from delivered source and SQL.

2. Background and related technical foundations

NASA POWER supplies analysis-ready daily meteorological data, including mean, maximum and minimum temperature, with an explicit time-standard choice [1]. This application preserves Local Solar Time rather than relabeling it as Korean Standard Time. Its coordinates denote requested points; the values are historical grid-based reanalysis rather than urban station readings. The source version therefore belongs in the analytical evidence, alongside query parameters.

Relational aggregation and OLAP summarize daily facts while preserving declared dimensions. MariaDB's WITH ROLLUP provides a concrete hierarchy of subtotals; its window functions calculate ranks and ordered frames without collapsing each daily row [2,3]. ROWS frames count records, which requires either validated continuous dates or additional gap detection before describing a frame as calendar days.

PDO prepared statements separate user values from statement structure, but identifiers and frame expressions still require fixed server-side choices [4]. Transactions provide a different guarantee: related parent and association writes must either commit together or be rolled back after an actual database failure [5]. Rejecting an invalid form before SQL runs is useful validation but is not evidence of rollback.

These documented mechanisms provide the technical foundation for the project. We do not claim a systematic literature review or a performance comparison with other climate portals. Our evaluation focuses on implementation correctness, source traceability and independent expected outputs under the project's bounded data volume.

3. Problem statement and data model

For a daily location record, define the daily temperature range as maximum minus minimum. A valid day has all three temperature values. A matching day is valid and satisfies both user-selected inclusive conditions: maximum temperature at or below a limit, and daily range at or below another limit. The matching rate is matching days divided by valid days, multiplied by 100. A zero valid-day denominator produces no rate and is excluded from ranking.

Dates are restricted to 2023-01-01 through 2025-12-31. There are 1,096 dates per requested point, including February 29, 2024. Mean, maximum and minimum values are stored as nullable DECIMAL columns. Missing values would remain NULL; the acquired source version contains no missing temperature rows or dates. All five source hashes exactly match the corresponding supplied reference hashes.

The schema contains locations, daily_temperatures, solar_terms, monthly_food_guides, saved_plans and plan_locations. The weather primary key is location/date. A plan/location association has a composite key and foreign keys to both parents; deleting a plan cascades its associations. Pairwise temperature-order CHECKs, unique monthly ingredient keys and date/range checks add database-level protection. Every table uses InnoDB and utf8mb4_unicode_ci.

The reference sources have different grains. A solar-term row is an actual occurrence date, with its name allowed to recur across years. A food row is a month/name editorial item. Neither is a foreign-key parent of each weather record. This design prevents a multi-item monthly guide from multiplying weather counts and avoids inventing relationships solely to increase table count.

4. Proposed analytical approach

A shared CTE selects the requested period and locations and assigns valid, matching and exceedance flags. Thresholds classify rows; they do not remove otherwise valid candidate days. Grouped queries use conditional averages and explicit flag sums, maintaining the same valid-day set for every temperature statistic. Annual and subtotal means come directly from daily values, rather than an unweighted average of monthly means.

Complex grouping supports location/year/month and location/year dimensions and returns valid counts, means of each daily temperature, mean daily range, period extremes, exceedance days, matching days and matching rates. Hierarchical exploration changes the actual SQL between year, month and day views. Common dates and drill bounds are intersected, and upward navigation restores the broader aggregation. WITH ROLLUP adds monthly, annual, location and all-location totals. Combined totals explicitly use location-days.

Ranking proceeds in stages: region counts, minimum-valid-day qualification, a high-precision DECIMAL rate, DENSE_RANK ordered only by that rate, and an outer Top N rank predicate. A region ID appears only in the final presentation order. Thus equal ratios receive equal ranks and Top 1 may legitimately return multiple locations. Rates are rounded only for display, after ranking.

Windowing derives a validated metric from mean, maximum, minimum or daily range. An internal query scans as far as 29 days before the displayed start, partitions by location and orders by date. The 7-day and 30-day ROWS frames use 6 and 29 preceding records. A valid-value count and date-span check must both confirm a complete window before its average is shown. Only the outer query trims the display range, preserving history across month/year boundaries and preventing the display filter from altering the same-date result.

These mechanisms keep numerical analytics in MariaDB. PHP handles validation, prepared execution, Session state and escaped HTML presentation. A Python Decimal program supplies an independent raw-data oracle for verification only; it is neither a web runtime dependency nor a substitute for the SQL calculations.

5. Implementation

The application contains nine user-facing pages and a separate POST action handler. A common bootstrap starts Session before output and loads centralized configuration and PDO. PDO uses utf8mb4, exception mode, associative results and native prepares. The connection's SQL mode includes ONLY_FULL_GROUP_BY; no global server mode is weakened to make a query run. Selected IDs generate one placeholder each, while SQL expressions come from server-side whitelists.

Common form inputs are strict real dates, one or more known IDs and finite decimals with two-place precision and bounded ranges. A valid request updates Session only after common and page options both pass. Invalid forms return an explanatory error and keep prior Session conditions. The controls include dates, numbers, checkboxes, select lists, reset and apply buttons. Core behavior works without JavaScript; optional confirmation is a local script.

Saved-plan creation inserts a parent and all selected associations in one transaction. Update locks and verifies the parent, updates its body, deletes old associations and inserts the replacement set before committing. Any exception rolls back the whole operation. Delete relies on the parent foreign key's cascading associations. An unchanged update is accepted after an explicit existence query; affected-row count is not misused as an existence test. Success uses Post/Redirect/Get.

Mutation forms require POST and a Session-bound CSRF token. Names and memos use native bound parameters and escaped HTML output; Unicode length checks count characters. Applying a plan validates stored common conditions and whitelists the destination analysis. Cookies are HttpOnly and SameSite=Lax; the interface clearly describes plans as shared demonstration records rather than private per-user data.

The seasonal page uses two independent prepared SELECTs. Calendar terms use a half-open date interval for the selected year/month; ingredients use month number. Source links and reference limitations are visible. KASI/RDA values were adopted from the supplied verified appendices, rather than newly downloaded here. No final official almanac check is claimed. The delivered site bundles CSS, JavaScript and its licensed Korean font, and it performs no external request to initialize data.

A revision corrected three observable persistence/UI defects: region-only updates now explicitly advance the parent timestamp; memo "0" is rendered literally; Save conditions validates and forwards the complete current form before storing a permanent plan. Failed saves preserve prior Session conditions. A compact blue/white original interface exposes seven primary comparison columns and retains every remaining statistic in an expandable panel. Its original SVG chart uses SQL results directly, preserves date spacing/NULL gaps and finite single-day points, and does not recompute moving averages.

6. Experiments and user scenarios

Evaluation combined actual-data oracle comparisons, transactional synthetic fixtures, real HTTP requests, Chromium interaction and clean installation. The raw-data oracle independently parses the five preserved NASA CSVs and computes Decimal statistics. Hashes and collection timestamps bind the expected answers to the acquired source version. Fixtures used for tie or sequence boundaries remain in transactions or temporary databases and are absent from the final seed/dump.

6.1 Actual-data comparison and tied ranking

For August 2024, maximum≤28°C and daily range≤10°C, each location has 31 valid days. Busan and Daegu each have 2 matching days and rank 1 at 6.45%; Jeju has 1 and rank 2 at 3.23%; Seoul and Gwangju have 0 and rank 3. The database, independent oracle, HTTP output and Chromium page agree. Top 1 returns both tied first-place locations. Figure 1 is an actual result-table capture.

Rank	Location	Matching days	Valid days	Matching rate	Selected period · LST
1	**Busan**	2	31	**6.45%**	2024-08-01 → 2024-08-31
1	**Daegu**	2	31	**6.45%**	2024-08-01 → 2024-08-31
2	**Jeju**	1	31	**3.23%**	2024-08-01 → 2024-08-31
3	**Seoul**	0	31	**0.00%**	2024-08-01 → 2024-08-31
3	**Gwangju**	0	31	**0.00%**	2024-08-01 → 2024-08-31

Figure 1. Actual August 2024 dense ranks with visible denominators.

Additional fixtures test 2/4 versus 3/6, zero-valid exclusion, minimum-valid qualification and near-equal rates 1/1095 versus 1/1096. The last two rates round to the same displayed percentage but retain distinct database ranks. This guards against an implementation that ranks pre-rounded percentages. Changing thresholds also preserves valid-day denominators while changing matches.

6.2 Grouping, hierarchy and continuous history

January–February 2024 grouping yields 60 valid days per location and a grand total of 300 location-days. All requested statistics were compared with independent daily expectations; means agreed within 0.00001°C. Seoul's partial-period mean is 0.018667°C, calculated from 31 January and 29 February rows. Hierarchical tests returned 5 year groups,10 month groups and 155 January location-day records. A partial January intersection contained 17 Seoul records and did not widen the user's dates.

For a synthetic sequence 1..35, day 7 has a 7-day mean 4, day 30 a 30-day mean 15.5, and day 31 has averages 28 and 16.5. Starting the display on day 31 leaves those results unchanged. Initial incomplete windows are NULL, and location partitions remain separate. A missing date or invalid day prevents a complete-frame average.

The actual Seoul record on January 1, 2024 has mean-temperature moving averages 1.141429°C over 7 days and 0.301333°C over 30 days; the interface displays 1.14 and 0.30. These values require December 2023 history. Forty raw-data cases span five locations, two boundary dates, four metrics and both frame outputs, including March 1 after leap February. Figure 2 shows the actual cross-year result.

Location	Date · LST	Mean °C	7 day mean °C	Valid / 7	30 day mean °C	Valid / 30	Matches limits
Seoul	2024-01-01	1.72	1.14	7 / 7	**0.30**	30 / 30	Yes

Figure 2. Seoul January 1, 2024 with continuous 7/30-day history.

6.3 Transactions and hostile requests

Committed create/read, same-value update, replacement locations and cascade delete passed CLI and web checks. An actual parent INSERT followed by a child FK failure left no parent or earlier child. An update followed by old-child deletion and a failing replacement insert restored the exact original body and associations. These failures originated at a CLI repository boundary, with no web failure switch.

Real HTTP tests stored a title containing SQL metacharacters as literal text; the location table and 5,480 weather records survived. A script-like memo was escaped. GET mutation requests, invalid CSRF, malformed IDs, unknown options and overlength Unicode input were rejected. An extra monthly food row left all weather summaries, ranks and windows unchanged. Chromium exercised the full comparison and plan workflow at 1440×900, 1366×768, 390×844 and 360×844 viewports across all nine screens plus valid edit, with zero unexpected console errors and no page overflow. Keyboard and no-JavaScript saving also passed.

6.4 Reinstallation and offline reproducibility

Fresh development containers used PHP 8.2.35, Apache 2.4.69 and MariaDB 10.4.34. A separately installed official XAMPP Linux 8.2.12 environment used PHP 8.2.12, Apache 2.4.58 and MariaDB 10.4.32. The actual XAMPP test copied source into `/opt/lampp/htdocs/team05`, used unchanged team05 defaults and ran with no container network. Both CLI and Apache SAPI/extension configurations were checked.

Create/insert, all nine screens and HTTP CRUD, every analysis/rollback suite, drop/create/insert and dump-only restoration passed. The dump was generated by an actual MariaDB client and imported by XAMPP's bundled mysql client into zero project tables. Ordered row hashes, column metadata, constraints and indexes matched the baseline after restoration. Tests used disposable containers and preserved the primary development DB. These observations establish repeatable functionality in the tested environments, not portability to every untested operating system.

6.5 Evidence integrity and artifact identity

Regression fixtures cover normal, failing and missing evidence, including reproduction of the former generator's misleading PASS behavior. The revised generator checks actual counts, suite rows, execution time, command, environment and source hashes. Missing or stale input yields NOT_RUN or PARTIAL; strict mode exits nonzero. The original evidence remains as historical records. Documentation generation is separately timestamped and does not impersonate test execution.

A committed-source ZIP is deployed into a separate Apache document root. File hashes measured inside the serving container, HTTP asset hashes, screenshot metadata and the captioned browser recording identify the same application source commit and manifest. Later document/package commits have a separate identity; equality concerns the executed application bytes rather than a recursive whole-archive hash. The full ZIP checksum is an external sidecar. All twelve English slides are actually rendered through LibreOffice to PDF and individual pages for visual inspection; opening a PPTX alone is not treated as rendering.

6.6 Equal assigned responsibility

Responsible member	Assigned module and paper section	Technical/report/paper/presentation share	Content slides
Nunnalin Heimvichit (2371001)	Data, schema, SQL files, reproducibility	20% in each	2–3
Jieun Hwang (2376329)	PHP/PDO/Session, filters/UI, seasonal references	20% in each	4–5
Sumin Cho (2376274)	Saved plans, CRUD and transaction failures	20% in each	6–7
Koo Daon (2476009)	Complex grouping and OLAP	20% in each	8–9
Jihee Choi (2276337)	Ranking, windows and chart	20% in each	10–11

Slides 1 and 12, report integration, English editing, PPT/video and final checks are shared equally. For an agreed total T, each person receives approximately T/5. These are confirmed responsibilities, while actual individual review, oral delivery and course submission remain NOT_RUN. One sequential Codex session implemented and verified the revision; existing Git authorship was preserved.

7. Discussion and limitations

The fixed dataset contains only 5,480 weather records, sufficient for this project's analytical correctness goals but insufficient to substantiate claims about distributed scale. No load benchmark, comparative latency experiment, usability study or measured recommendation accuracy was performed. The system interprets historical matching rates, rather than future forecasts or validated comfort/safety scores.

Grid resolution and the distinction between requested coordinates and observation stations limit city-specific interpretation. LST daily dates also differ from exact KST calendar-transition times. The KASI and RDA snapshot provenance is explicit; this implementation did not independently download their full pages or compare dates with the final official almanac. Ingredient guidance remains an editorial reference.

The application intentionally omits authentication, and plans are shared demonstration records. Windows/macOS and the professor's physical host were not tested. Human contribution declarations and final course submissions remain subject to team review. Initial credential failures were resolved by user authentication; the original history/review tag and validated revision commits are now preserved on GitHub. Current synchronization is documented with observed SHAs. These limits are retained in the requirement matrix rather than hidden behind a broad “all requirements complete” statement.

8. Conclusion

Seasonal Living Guide integrates four distinct database analysis modes with persistent comparison plans and independent seasonal references. Explicit validity rules preserve denominators; high-precision dense ranking preserves ties; lookback-first windowing preserves cross-boundary history; and actual FK-failure tests demonstrate atomic parent/association writes. Independent source-based expectations and fresh offline XAMPP

restoration provide concrete evidence of correctness and reproducibility for the implemented scope. Actual member sign-off, oral delivery and course submission remain separate from the verified technical draft.

References

[1] NASA POWER. Daily API documentation.

[2] MariaDB. Window Functions Overview.

[3] MariaDB. SELECT WITH ROLLUP.

[4] PHP. PDO::prepare.

[5] PHP. Transactions and auto-commit.

[6] KASI. Calendar reference, 2023, with corresponding 2024/2025 links preserved in the source snapshot.

[7] RDA. Monthly food guide source family; exact historical feature URLs and dates in the delivered curated CSV.

[8] Apache Friends. XAMPP distribution. Tested official archived Linux 8.2.12 installer hash is retained in project evidence.

[9] Project evidence. `data/manifest.json`, `docs/evidence/revision-2026-11-12/oracle.json`, analysis/transaction test JSON files, browser checks, clean-install.json and xampp-install.json. Draft source and executable implementation are included in the same review package.